

Exercise - 3 Solutions

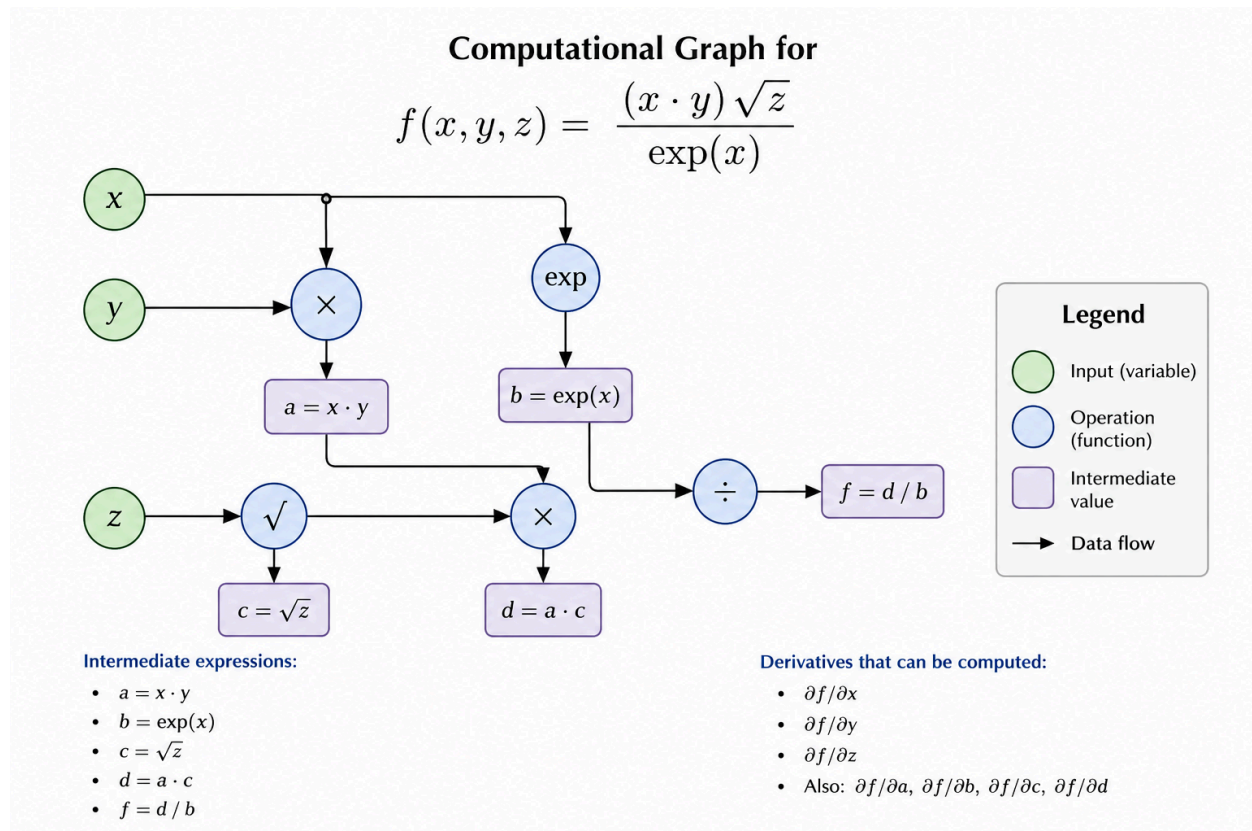
3.1 Computational Graphs

Draw the computational graph for the function:

$$f(x, y, z) = \frac{(x \cdot y) \sqrt{z}}{\exp(x)} \quad (1)$$

Write down some of the derivatives that you could compute with this graph (you do not need to compute the derivatives, just discuss which ones could be calculated and why).

→



3.2 Backpropagation

Explain the purpose of the backpropagation algorithm

→

Backpropagation is an efficient algorithm for computing gradients of a function with respect to all model parameters.

In machine learning, the model produces predictions, and a **loss function** measures the prediction error. To improve the model, we need the gradient of the loss with respect to every parameter.

Backpropagation works by:

- Performing a forward pass to compute the loss.
- Starting from the loss and propagating derivatives backward.
- Applying the chain rule at each operation.
- Accumulating gradients for all parameters.

These gradients are then used by optimization algorithms such as gradient descent to update the model.

3.3 Gradient Descent

Explain the idea behind the gradient descent algorithm. Write down the update formula. What would happen if the sign was inverted?

→

Gradient descent is an optimization method used to minimize a loss function by iteratively adjusting parameters in the direction of decreasing loss.

Update Formula

$$w_{t+1} = w_t - \alpha \nabla L(w_t)$$

Where:

- w_t = current parameter vector
- α = learning rate
- $\nabla L(w_t)$ = gradient of the loss

Intuition

The gradient points in the direction of the steepest increase of the loss. By subtracting the gradient, we move in the direction of the steepest decrease.

What Happens if the Sign Is Inverted?

If the update rule becomes:

$$w_{t+1} = w_t + \alpha \nabla L(w_t)$$

The algorithm performs gradient ascent instead of gradient descent.

Consequences:

- The loss increases rather than decreases.
- Training diverges.
- The model becomes worse over time.

3.4 Dataset Exploration

Load the dataset and explore it. —

1. How many images are in the training and test splits?

→ (Ref: ipynb Notebook)

2. What is the class distribution for each label? Are the classes balanced?

→ (Ref: ipynb Notebook)

3. Display a few example images for each label combination. What patterns do you notice?

→ (Ref: ipynb Notebook)

OBSERVATIONS:

- The dataset contains CARLA front-camera images with three Boolean labels: pedestrian presence, traffic light presence, and vehicle presence.
- The number of images was obtained by counting the rows in the training and test label files.
- The class distributions show that the dataset is not perfectly balanced. Vehicles appear frequently in many driving scenes, while pedestrians and traffic lights appear less often. This imbalance may make pedestrian and traffic light detection more difficult.
- The displayed images mainly show urban driving situations. Vehicles are commonly visible on the road, traffic lights usually appear near intersections, and pedestrians appear less frequently, often near sidewalks or crossings.
- The dataset mainly represents clear daytime driving conditions. From a safety perspective, this means that the dataset only supports claims for this limited Operational Design Domain. It does not provide strong evidence for performance in night, fog, rain, or other unseen conditions. This is related to hazard H-4, where autonomous mode remains active outside the intended ODD.

3.5: Train Three Binary Classifiers

Train three separate binary classifiers on the training split – one for each detection task (pedestrian, traffic light, vehicle). Use Python and preferably PyTorch. You may use a pre-trained backbone (e.g., ResNet-18) and fine-tune it for binary classification. Commit your code to your git repository.

(Ref: ipynb Notebook)

1. Describe your model architecture and training setup (optimizer, learning rate, loss function).

→

Use ResNet-18 as a pre-trained backbone.

The final layer is replaced by a single output neuron because each task is binary classification.

Training setup:

Backbone: ResNet-18 -- is a lightweight and well-established convolutional neural network for image classification. Using a pre-trained backbone allows the model to reuse visual features learned from ImageNet, which improves convergence and reduces training time.

Loss function: Binary Cross Entropy with Logits Loss -- it is appropriate for binary classification and combines sigmoid activation with binary cross-entropy loss in a numerically stable implementation.

Optimizer: Adam -- it usually works well for fine-tuning neural networks and adapts the learning rate for each parameter.

Learning rate: 0.0001 -- A small learning rate of 0.0001 is used to avoid destroying the useful pre-trained features too quickly.

Batch size: 32 -- provides a reasonable balance between stable gradient updates and GPU memory usage.

Epochs: 5 -- The number of epochs was kept small to focus on training baseline models rather than maximizing performance.

2. Train each model and plot the training and validation loss curves. Do the models converge?

(Ref: ipynb Notebook)

→

The models partially converge:

- Vehicle and traffic light detectors show reasonable convergence behavior. The pedestrian detector overfits and does not generalize well.
- This demonstrates that convergence should not only be judged by training loss but also by validation performance.

Vehicle Detector --

The vehicle detector shows strong learning behavior. The training loss decreases steadily from approximately 0.31 to below 0.10, indicating that the model successfully learns vehicle-related visual features.

However, the validation loss remains relatively high and slightly increases after later epochs. This suggests that the model begins to overfit the training data while generalization performance stops improving.

Overall, the model converges on the training set but shows mild overfitting.

Traffic Light Detector --

The traffic light detector shows the best overall behavior among the three models. Both training and validation losses remain relatively low throughout training.

The training loss decreases steadily, while the validation loss fluctuates only slightly and stays stable overall. This indicates that the model generalizes reasonably well to unseen data.

Therefore, the traffic light detector can be considered to have converged successfully without severe overfitting.

Pedestrian Detector --

The pedestrian detector behaves differently from the other two models. While the training loss decreases steadily, the validation loss continuously increases during training.

This indicates significant overfitting: the model memorizes patterns from the training data, but fails to generalize to unseen pedestrian examples.

This behavior is likely caused by:

- Class imbalance, fewer pedestrian samples, and greater visual variability of pedestrians.
- From a safety perspective, this is especially important because pedestrian detection is directly related to hazard H-1 identified in the STPA analysis.

3. Why might training three separate models be preferable to a single multi-label classifier from a safety perspective?

→

From a safety perspective, training three separate models can be preferable because each safety-critical perception task can be analyzed independently.

Each detector corresponds to different hazards identified earlier:

pedestrian detection → H-1,
traffic light detection → H-2,
vehicle detection → H-3.

Following are two reasons to use separate models.

Failure Isolation – A failure in one detector does not directly affect the internal behavior of the other detectors.

Independent Safety Evaluation – Each detector can be tested with task-specific metrics such as recall, false negatives, and precision.

3.6: Evaluation

Evaluate each of your three trained models on the test split.

1. Report accuracy, precision, recall, and F1-score for each model.

→ (Ref: ipynb Notebook)

Model	Accuracy	Precision	Recall	F1-score
Pedestrian Detector	0.790000	0.452652	0.338527	0.387358
Traffic Light Detector	0.936667	0.941860	0.971749	0.956571
Vehicle Detector	0.886111	0.955812	0.889259	0.921335

2. Which model performs worst? Hypothesize why.

→

The **pedestrian detector** performs worst among the three models.

ModelRecall:

Pedestrian Detector - **0.3385**

Traffic Light Detector - **0.9717**

Vehicle Detector - **0.8893**

The pedestrian model has:

the lowest recall, the lowest F1-score, and the lowest precision.

This means the model struggles both to correctly detect pedestrians and to avoid incorrect pedestrian predictions.

Several reasons likely contribute to this poor performance:

Class imbalance

Pedestrians appear less frequently in the dataset compared to vehicles and traffic lights. The model therefore has fewer examples to learn from.

Visual variability

Pedestrians vary strongly in: pose, clothing, size, orientation, and occlusion.

This makes pedestrian detection more difficult than detecting large vehicles or structured traffic lights.

Small object size

Pedestrians often occupy only a small region of the image, especially when far away from the ego vehicle.

Overfitting behavior

During training, the pedestrian model showed increasing validation loss while training loss decreased, indicating overfitting and weak generalization.

From a safety perspective, this result is especially concerning because pedestrian detection failures directly relate to hazard H-1 identified in the earlier STPA analysis.

3. From a safety perspective, which metric matters most for each model – precision or recall? Explain.

→ From a safety perspective, recall is the most important metric for all three models, especially for pedestrian detection.

Pedestrian Detector

For pedestrian detection, recall is critical because:

- A false negative means the model misses a real pedestrian, which could directly lead to collisions or fatalities.
- Although precision is also relevant, false positives usually lead only to unnecessary braking, which is safer than missing a pedestrian entirely.
- Therefore, recall is the most safety-critical metric for pedestrian detection.

Traffic Light Detector

For traffic light detection, recall is also highly important.

- Missing a traffic light, especially a red light, may cause:
- unsafe intersection crossing, collisions, or traffic violations.

- False positives may cause unnecessary stopping, but false negatives are generally more dangerous.

Vehicle Detector

For vehicle detection, recall again matters most because:

- failing to detect another vehicle may lead to unsafe lane changes or collisions.
- High precision is desirable to reduce unnecessary braking or warnings, but minimizing false negatives remains the higher safety priority.

Saved all three trained models. Can use them in all subsequent exercise sheets!!

3.7: ODD Gap Analysis Comparing the ODD to the Training Data

In Exercise Sheet 2 you defined an Operational Design Domain (ODD) based on the system description – before seeing any data. Now that you have explored the dataset and trained your models, revisit that ODD.

1. Look at the data you trained on. What conditions does it cover?
List the environmental conditions represented in the training set (weather, lighting, scene types, etc.).

→

ODD dimension	Conditions represented in training data
Weather	Mostly clear weather. No visible rain, fog, snow, or hail.
Lighting	Daytime scenes with normal sunlight.
Camera condition	Forward-facing camera view, clean and fixed perspective.

Scene type	Urban CARLA road scenes with lanes, intersections, vehicles, pedestrians, and traffic lights.
Vehicle speed	Appears to represent city-driving / intersection situations, but exact speed is not directly available from the labels.

– The training data mainly covers **clear daytime urban driving** with a clean front-facing camera. It includes roads, intersections, vehicles, pedestrians, and traffic lights. The dataset does not visibly include difficult environmental conditions such as night, fog, heavy rain, snow, glare, or dirty camera images.

2. Compare this to the ODD you defined in Exercise Sheet 2. Which ODD dimensions are not covered by the training data?

→

ODD dimension	Defined as valid in Exercise 2	Covered by training data?	Missing / weakly covered conditions
Weather	Clear, cloudy, no precipitation	Partly covered	Clear weather is covered, but cloudy variation is not clearly proven. Rain, fog, snow, hail are not covered.
Lighting	Daytime with normal sunlight	Covered	Night, dusk/dawn, tunnels, glare, and low-light cases are not covered.
Camera condition	Forward-facing, clean, fixed mount	Covered	Dirty lens, wet lens, obstructed view, tilted camera, blur are not covered.
Scene type	Urban roads with structured lanes	Mostly covered	Highways, off-road, rural roads, construction zones, unusual road layouts are not covered.
Vehicle speed	Low speed / intersection approach < ~50 km/h	Not directly verifiable	Speed metadata is not visible from the label file, so speed coverage cannot be strongly claimed.

3. What are the implications of this gap for your models' safety? For which ODD conditions can you not make any claims about model performance?

→

Gap	Safety implication	Related hazard from Exercise 2
No rain/fog/snow examples	Perception models may fail when visibility changes or objects become harder to see.	H-4: Autonomous mode active during ODD violation
No night or low-light examples	Pedestrians, vehicles, and traffic lights may be missed in darkness.	H-1, H-2, H-3, H-4
No glare or tunnel lighting	Camera images may be overexposed or underexposed, reducing detection quality.	H-1, H-2
No dirty/wet/blurred camera cases	The perception system may silently fail if the camera input quality drops.	H-4
Limited scene diversity	The model may not generalize to highways, construction zones, rural roads, or unusual intersections.	H-2, H-3, H-4
No direct speed metadata	It is difficult to verify whether the model was tested only within the intended low-speed ODD.	H-4

4. Note down the gaps you identified (if any). You will revisit them in Sheets 4 and 5 when you formally measure ODD coverage and refine the ODD.

→

Gap ID	Identified gap	Why it matters
G-1	Missing adverse weather: rain, fog, snow, hail	Objects may become harder to detect.
G-2	Missing night, dusk/dawn, tunnel, and glare conditions	Lighting changes can reduce camera-based detection performance.
G-3	Missing dirty, wet, obstructed, blurred, or tilted camera cases	Camera quality problems can cause perception failure.
G-4	Limited scene diversity beyond urban CARLA roads	Model may not generalize to highways, rural roads, or construction zones.

G-5	Vehicle speed not directly represented in the labels	Cannot prove that the dataset only covers the intended low-speed ODD.
G-6	Pedestrian examples are weaker than vehicle/traffic-light examples	Pedestrian detector safety is limited, especially for H-1.

→

Question	Summary answer
What does the data cover?	Clear daytime urban CARLA driving with a clean front-facing camera.
What is missing?	Rain, fog, snow, night, glare, dirty camera, highways, construction zones, and speed metadata.
Safety implication	The model cannot be considered reliable outside clear daytime urban conditions.
Main related hazard	H-4: Autonomous mode remains active outside the ODD.